

UD 6. Prácticas

Índice de contenidos

Prácticas.....	2
Práctica 1: Consumir REST API con GET: “zippopotam.us”	2
Práctica 2: Consumir REST API con GET: “europeana”	2
Práctica 3: Consumir REST API con GET: “PokéAPI”	3
Práctica 4: Consumir REST API con POST: “Productos”	4
Práctica 5: Consumir REST API con PUT: “Aumentar stock Productos”	5
Práctica 6: Implementar REST API con GET: “Biblioteca”	6
Práctica 7: Implementar REST API con POST: “Biblioteca”	9
Práctica 8: Implementar REST API con PUT: “Biblioteca”	10
Práctica 9: Implementar REST API con DELETE: “Biblioteca”	11
Práctica 10: Implementar REST API con PUT: “Tweetfony”	12

Prácticas

Práctica 1: Consumir REST API con GET: “zippopotam.us”

Crea un nuevo método en el controlador. Este método deberá realizar una petición GET para obtener información de un pueblo a partir de su código postal. Mostrar únicamente el nombre del pueblo en la plantilla.

Usa la siguiente REST API: <https://api.zippopotam.us/>

Práctica 2: Consumir REST API con GET: “europeana”

Para esta práctica se usará la API pública de “europeana”. Esta API contiene una gran base de datos sobre obras de arte.

Para empezar, deberás registrarte para obtener una api-key en la siguiente página: <https://pro.europeana.eu/pages/get-api>. Te llegará la api-key a tu email.

La documentación sobre los parámetros de una petición junto a algunos ejemplos se encuentran en <https://pro.europeana.eu/page/search>.

Con el parámetro query “**wskey**” podrás indicar la api-key en la petición.

Se pide:

- Crear un controlador llamado `EuropeanaApiController`.
- Crear un método en el controlador llamado **showArtWorks**. La ruta será en: `/europeanaapi/buscar/{nombre_obra}` donde {nombre_obra} es el nombre de la obra de arte que buscaremos.
- Este método deberá realizar una petición GET a la API, y buscar todas las obras de arte cuya información contenga el {nombre_obra} que se indicó en la ruta. No te olvides del parámetro wskey.
- Una vez obtenidas todas las obras de arte (pueden ser muchas), muéstralas todas en una plantilla. Por cada obra de arte se deberá mostrar: **Título, Autor, Descripción, Museo en el que se encuentra, Imagen**. Prueba primero la petición GET en POSTMAN para averiguar el formato de la respuesta y encontrar esos datos (título, autor...). Si te rindes, aquí tienes la respuesta (INTÉNTALO POR TI MISMO PRIMERO, EN EL EXAMEN NO TE DARÉ ESTO):

Práctica 3: Consumir REST API con GET: “PokéAPI”

Para esta práctica se usará la API pública de “PokéAPI”. Esta API contiene una gran base de datos sobre Pokémon: habilidades, objetos...

La documentación sobre los parámetros de una petición para ver información de un Pokémon se encuentra en: <https://pokeapi.co/docs/v2#pokemon>

Se pide:

- Crear un controlador llamado `PokeApiController`.
- Crear un método en el controlador llamado **`verInfoPokemon`**. La ruta será en: **`/pokeapi/buscar/{nombre_pokemon}`** donde `{nombre_pokemon}` es el nombre del pokemon que buscaremos.
- Este método deberá realizar una petición GET a la API, y buscar la información del Pokémon cuyo nombre sea `{nombre_pokemon}` que se indicó en la ruta. Si quieres probar la API y eres tan poco friki que no te sabes ningún nombre de Pokémon (¿qué haces en un ciclo de informática?), te doy algunos: pikachu, charizard, piplup.
- Una vez obtenida la información, debes mostrarla en una plantilla. Solo se debe mostrar: **Habilidades, Movimientos, Tipos**. Ejemplo de cómo se debe mostrar:

Información de Charizard

Tipos:

- fire
- flying

Habilidades:

- blaze
- solar-power

Movimientos:

- mega-punch
- fire-punch
- thunder-punch
- scratch
- ...

Práctica 4: Consumir REST API con POST: “Productos”

En esta práctica mejoraremos la actividad 6.1 de la unidad. En esa actividad, se hizo una petición POST a la API <https://restful-api.dev/> para crear un nuevo producto (en ese caso una tostadora). El problema es que teníamos que escribir los datos del producto (nombre, marca, color) manualmente en el código. El objetivo de esta práctica es permitir que el usuario pueda introducir los datos a través de un formulario. Una vez se tengan los datos, se enviarían a la API con una petición POST. Para ello:

- Crea un formulario con los campos: nombre, marca y color. Este se mostrará en la ruta **/crear/producto**.
- Al enviar el formulario, se obtendrán los datos introducidos (validándolos previamente) y se enviarán a la API mediante una petición POST para almacenarlos.

Práctica 5: Consumir REST API con PUT: “Aumentar stock Productos”

En esta práctica se utilizará la API <https://restful-api.dev/>

Crea un método en la ruta `/aumentarstock/{id}` que aumente en una unidad el stock de un producto:

- En primer lugar se realizará una petición **GET** para obtener la información del producto con el id {id}.
- Si el id indicado no se corresponde con ningún producto, lo creará (mediante una petición **POST**) y mostrará una plantilla con el id del producto creado. Ejemplo del cuerpo enviado para la creación del producto:

```
{
  "name": "Cafetera",
  "data": {
    "marca": "Samsung",
    "color": "azul",
    "stock": 0
  }
}
```

- Si sí existe, aumentará en una unidad el stock (mediante una petición **PUT**) y mostrará en una plantilla el nuevo stock junto a la fecha y hora de actualización.

Práctica 6: Implementar REST API con GET: “Biblioteca”

En estas prácticas y en las siguientes utilizaremos el proyecto de Laravel “**biblioteca-laravel**”. Solo tiene dos entidades: Biblioteca y Libro. Una biblioteca puede contener muchos libros, pero un libro solo puede pertenecer a una única biblioteca.

Puedes descargar el proyecto desde Moodle. Una vez descargado, sitúate con la terminal en el proyecto y realiza lo siguiente:

1. Ejecutar **composer install** para instalar las dependencias software.
2. Personalizar la configuración de la base de datos en el archivo **.env**.
3. Ejecutar **php artisan migrate** para crear las tablas en la base de datos.
4. Ejecutar **php artisan db:seed** para cargar los datos de ejemplo en la BD.

Se pide crear una API REST para la gestión de las bibliotecas y libros de este proyecto.

Todos los recursos estarán dentro de la ruta **api/** del siguiente modo:

- Bibliotecas: **/api/bibliotecas**
- Libros: **/api/libros**

Crea un controlador llamado BibliotecaApiController. Todos los métodos que implementarás en esta práctica y las siguientes irán en este controlador.

Crea un método GET en la ruta **/api/bibliotecas/{id}** que devuelva la información de una biblioteca. Deberá devolver el resultado en el siguiente formato exactamente:

```
{
  "nombre": "Biblioteca Municipal de Arahal",
  "direccion": "Avenida de María Luisa Num 8",
  "horario": "De Lunes a Viernes de 9:00 a 20:00",
  "libros": [
    1,
    2,
    3
  ]
}
```

Crea un método GET en la ruta **/api/bibliotecas** que devuelva las id de todas las bibliotecas. Deberá devolver el resultado en el siguiente formato exactamente:

```
{
  "bibliotecas": [
    1,
    2
  ]
}
```

Crea un método GET en la ruta **/api/libros/{id}** que devuelva la información de un libro. Deberá devolver el resultado en el siguiente formato exactamente:

```
{
  "titulo": "El Quijote",
  "autor": "Miguel de Cervantes",
  "numPaginas": 700,
  "disponible": "sí",
  "biblioteca": 1
}
```

Crea un método GET en la ruta **/api/libros** que devuelva los id de todos los libros. Deberá devolver el resultado en el siguiente formato exactamente:

```
{
  "libros": [
    1,
    2,
    3,
    4,
    5
  ]
}
```

Crea un método GET en la ruta **/api/bibliotecas/buscar/nombre/{nombre}** que devuelva la información de una biblioteca que tenga exactamente ese nombre. El formato de resultado será igual que con el método **/api/bibliotecas/{id}**.

Crea un método GET en la ruta **/api/bibliotecas/buscar/horario/{horario}** que devuelva la información de una biblioteca que **contenga** el horario. Es decir, que si se hace “GET **/api/bibliotecas/buscar/horario/Lunes**”, se listarán todas las bibliotecas que contengan “Lunes” en su horario. El formato de resultado será igual que con el método **/api/bibliotecas/**, mostrando únicamente un listado de los id de las bibliotecas.

Crea un método GET en la ruta **/api/bibliotecas/buscar/libros/{id}** que devuelva los nombres de los libros que hay en la biblioteca cuyo id es {id}. Deberá devolver el resultado en el siguiente formato exactamente:

```
{
  "biblioteca": "Biblioteca Pública de Marchena",
  "libros": [
    "Lo que el viento se llevó: el libro",
    "El retrato de Dorian Gray"
  ]
}
```


Práctica 7: Implementar REST API con POST: “Biblioteca”

Crea un método POST en la ruta **/api/bibliotecas** que almacene una biblioteca.

Un ejemplo del formato de la petición que se recibiría por parte del cliente sería:

```
{
  "nombre": "Librería Lello",
  "direccion": "R. das Carmelitas 144, 4050-161 Porto, Portugal",
  "horario": "Lunes a viernes de 9:00-19:00"
}
```

Un ejemplo del formato de la respuesta con código 201 sería:

```
{
  "message": "Biblioteca con ID #3 almacenada correctamente"
}
```

Crea un método POST en la ruta **/api/libros** que almacene un libro.

Un ejemplo del formato de la petición que se recibiría por parte del cliente sería:

```
{
  "titulo": "El Señor de los Anillos",
  "autor": "J.R.R. Tolkien",
  "numPaginas": 1633,
  "disponible": "sí",
  "idBiblioteca": "3"
}
```

Un ejemplo del formato de la respuesta con código 201 sería:

```
{
  "message": "Libro con ID #6 almacenado correctamente"
}
```

En el caso de que no exista ninguna biblioteca con el id indicado, se devolverá la siguiente respuesta con código 409:

```
{
  "message": "Error al crear libro. No existe la biblioteca con ID #3. A continuación se listan las bibliotecas disponibles.",
  "bibliotecasDisponibles": [1, 2]
}
```

Práctica 8: Implementar REST API con PUT: “Biblioteca”

Crea un método PUT en la ruta `/api/bibliotecas/{id}` que permita modificar la información de una biblioteca.

Un ejemplo del formato de la petición que se recibiría por parte del cliente sería:

```
{
  "nombre": "Librería Lello",
  "direccion": "R. das Carmelitas 144, 4050-161 Porto, Portugal",
  "horario": "Lunes a viernes de 9:00-19:00. Sábados y domingos cerrados."
}
```

Un ejemplo del formato de la respuesta con código 201 sería:

```
{
  "message": "Biblioteca con ID #3 modificada correctamente"
}
```

Crea un método PUT en la ruta `/api/libros/{id}` que permita modificar la información de un libro.

Un ejemplo del formato de la petición que se recibiría por parte del cliente sería:

```
{
  "titulo": "El Señor de los Anillos",
  "autor": "J.R.R. Tolkien",
  "numPaginas": 1633,
  "disponible": "no",
  "biblioteca": 1,
}
```

Un ejemplo del formato de la respuesta con código 201 sería:

```
{
  "message": "Libro con ID #6 modificado correctamente"
}
```

Práctica 9: Implementar REST API con DELETE: “Biblioteca”

Crea un método DELETE en la ruta **/api/bibliotecas/{id}** que permita eliminar una biblioteca.

Si todo ha ido bien, no se devolverá ningún contenido (null) y se enviará el código 204.

En el caso de que no exista ninguna biblioteca con ese {id}, se deberá enviar la siguiente respuesta con código 404:

```
{
  "message": "La biblioteca con ID #7 no se ha podido borrar porque no existe."
}
```

Crea un método DELETE en la ruta **/api/libros/{id}** que permita eliminar un libro.

Si todo ha ido bien, no se devolverá ningún contenido (null) y se enviará el código 204.

En el caso de que no exista ningún libro con ese {id}, se deberá enviar la siguiente respuesta con código 404:

```
{
  "message": "El libro con ID #9 no se ha podido borrar porque no existe."
}
```

Crea un método DELETE en la ruta **/api/libros/nombre/{nombre}** que permita eliminar un libro a partir de su nombre.

Si todo ha ido bien, no se devolverá ningún contenido (null) y se enviará el código 204.

En el caso de que no exista ningún libro con ese {nombre}, se deberá enviar la siguiente respuesta con código 404:

```
{
  "message": "No existe ningún libro con ese nombre."
}
```

Práctica 10: Implementar REST API con PUT: “Tweetravel”

Crea la ruta `/api/users/{idUser}/tweets/{idTweet}/` para que puedas modificar si a un usuario le gusta o no un tweet, enviando mediante una petición PUT un parámetro llamado "like" con valores true o false.